ARQUITECTURA DE REDES I / ARQUITECTURA DE REDES



GP23

Práctica sobre el protocolo HTTP

Análisis de HTTP con Wireshark: mensajes, conexiones persistentes y no persistentes, GET condicional, cookies, autenticación y formularios.

Evaluable en PL3 + entrega de la memoria en Blackboard

Curso 2026-27

1. Presentación

En esta sesión analizarás con Wireshark cómo un navegador descarga páginas web mediante HTTP: qué mensajes envía, cómo usa las conexiones TCP, cómo evita descargar de nuevo lo que ya tiene y cómo se identifica ante el servidor. Trabajarás con tu propio navegador y con una captura de referencia que contiene todos los casos.

Objetivos

Al terminar la sesión deberías ser capaz de:

- Leer un mensaje HTTP: línea de petición y de estado, cabeceras principales, códigos de estado y dónde viajan los datos de un formulario.
- Interpretar un GET condicional (`If-Modified-Since`, código de respuesta 304) y explicar el papel de la caché del navegador y de un proxy.
- Explicar que HTTP no guarda estado y cómo lo suplen las cookies (incluida la cookie de sesión de un inicio de sesión por formulario) y la autenticación básica; interpretar 401, 403 y 303.
- Situar HTTP sobre TCP: resolución DNS y establecimiento previos, puertos y segmentación de un mensaje grande.

Cómo se evalúa

- **Entrega de la memoria** en Blackboard (actividad “GP23 - Entrega de la memoria”): una respuesta por cuestión, en el plazo que indique tu profesor.
- **Test** sobre la sesión (PL3).

Cómo trabajar

- Predice antes de mirar. Varias cuestiones te piden anticipar qué vas a ver: escribe tu predicción y después compruébala. Equivocarse no penaliza; es la mejor forma de detectar qué no tenías claro.
- Responde con tus palabras y justifica con la captura: número de trama, nombre del campo o de la cabecera y su valor.

Escenario

La asignatura dispone de un servidor web de prácticas que solo habla HTTP (`lab.redesuah.win`). Además, algunos de los objetos de las webs hospedadas en el servidor cargan imágenes de un segundo servidor (`images.redesuah.win`).

Nombre	Dirección IP	Qué sirve
`lab.redesuah.win`	157.230.93.197	Las páginas de la práctica (`http://lab.redesuah.win/`)
`images.redesuah.win`	68.183.129.29	Servidor web con imágenes

Trabajaremos de la siguiente manera:

- Visitarás con tu navegador tres páginas del servidor (cuestiones 1 a 8).
- Después, usarás la captura de referencia captura_HTTP_gp23.pcap, para tres casos que dependen del estado del navegador o del momento y que así verás todos igual (cuestiones 9 a 15).
- Por último, volverás a tu navegador para iniciar sesión por formulario y comparar lo que veas con las cookies y la autenticación de la referencia (cuestiones 16 y 17).

La captura de referencia se tomó en un equipo de la red (192.168.1.10, con su servidor DNS en 192.168.1.1) mientras un navegador visitaba, una tras otra, estas páginas (la numeración de las partes es la de la toma original, de la que se han quitado las que no se usan):

Parte	Desde (t ≈)	Página	Qué se observa	La trabajas en
1	0 s	`basico.html`	DNS, conexión, petición y respuesta básicas	Tu captura
2	15 s	`condicional.html` (y F5)	GET condicional	Referencia
3	34 s	`largo.html`	Respuesta en varios segmentos TCP	Tu captura
4	49 s	`objetos.html`	Objetos en dos servidores	Tu captura
7	111 s	`cookies/`	Cookies	Referencia
8	130 s	`protegido/` y `protegido/profesor/`	Autenticación: 401, 200, clave errónea y 403	Referencia
10	182 s	`formulario.html`	Formulario con GET y con POST, cookie de sesión, zona privada y salida	Tu captura

> **Sobre el servidor y la captura**
> Los navegadores actuales intentan conectar primero con la versión cifrada de una web (HTTPS) y, si no existe, vuelven a HTTP. Como en esta práctica queremos leer los mensajes HTTP, el servidor de prácticas no ofrece la versión cifrada. En la captura de referencia, las direcciones del equipo y de su red se han sustituido por direcciones de ejemplo.

Preparación (5 min)

- Abre Firefox. Pulsa F12, ve a la pestaña Red y marca “Desactivar caché” (déjala abierta mientras capturas). No uses una ventana privada.
- Abre Wireshark y empieza a capturar en la interfaz de red del equipo. Aplica el filtro de visualización `http`.
- Añade la columna `tcp.stream` (número de conexión TCP); el Anexo explica cómo. La necesitarás para saber qué mensajes van por la misma conexión.

2. Tu captura

Con “Desactivar caché” marcada y Wireshark capturando, visita estas páginas en este orden, sin cerrar el navegador. Puedes ir respondiendo las cuestiones mientras capturas, o capturar todo primero y guardar la captura (`Archivo → Guardar como`) para analizarla después.

- `basico.html`: **`http://lab.redesuah.win/lab/basico.html`**
- `largo.html`: **`http://lab.redesuah.win/lab/largo.html`**
- `objetos.html`: **`http://lab.redesuah.win/lab/objetos.html`**

Usa el filtro `http`, o `http.host == "lab.redesuah.win" || dns`, para quedarte con lo que te interesa. Si tu captura falla, responde con la parte correspondiente de la captura de referencia e indícalo.

Tu navegador

Cuestión 1.
Captura con tu navegador la descarga de `http://lab.redesuah.win/lab/basico.html`. ¿Qué versión de HTTP usa tu navegador?

Cuestión 2.
¿Cuál es la dirección IP de tu equipo y cuál la del servidor?

DNS, conexión y petición básica (`basico.html`)

Cuestión 3.
Antes del primer SYN hacia el servidor deberían aparecer dos mensajes que no son TCP. ¿Qué son y por qué tienen que ir antes? ¿Por qué no vuelven a aparecer al pedir otras páginas de `lab.redesuah.win`, pero sí al cargar `objetos.html`? Si en tu captura no aparecen, usa las tramas 1 y 2 de la captura de referencia.

Cuestión 4.
Compara el `Content-Length` de la respuesta con la longitud de la trama que la contiene. Descompón la longitud de la trama en cabecera Ethernet + cabecera IP + cabecera TCP + línea de estado y cabeceras HTTP + cuerpo.

Cuestión 5.
Compara las cabeceras `Last-Modified` y `Date` de la respuesta. ¿Cuándo se modificó el fichero por última vez? ¿Quién fija cada una de esas fechas?

Un documento largo (`largo.html`)

Cuestión 6.
¿En cuántos segmentos TCP viaja la respuesta y cuántos bytes de datos lleva cada uno? Comprueba que la suma es igual a la línea de estado y las cabeceras más los 4163 bytes del cuerpo. ¿En qué segmento va la línea de estado?

Cuestión 7.
Activa y desactiva el reensamblado de TCP (`Edit → Preferences → Protocols → TCP → "Allow subdissector to reassemble TCP streams"`). ¿Qué cambia en la lista de paquetes? ¿Sabe HTTP en cuántos segmentos viaja su mensaje?

Objetos en dos servidores (`objetos.html`)

Cuestión 8.
¿Cuántos GET hay, a qué servidores y por cuántas conexiones TCP? ¿Alguno reutiliza una conexión que ya estaba abierta? ¿Por qué aparece justo antes una consulta DNS?

3. La captura de referencia

Abre `captura_HTTP_gp23.pcap`. Estas tres partes dependen de lo que el navegador ya tiene guardado (copias en caché, cookies, contraseñas) o del momento en que se hace cada petición; en la captura de referencia todos veréis lo mismo. Para cada parte, filtra por el tiempo (por ejemplo, `frame.time_relative >= 15 && frame.time_relative < 34`) o localiza la página con el filtro `http`.

GET condicional (`condicional.html`)

> **Antes de abrir la parte 2**
> El navegador descarga `condicional.html` y, cuatro segundos después, el usuario recarga la página (F5). El navegador ya tiene una copia: ¿qué crees que le pedirá al servidor la segunda vez, y qué le responderá este si el fichero no ha cambiado?

Cuestión 9.
Antes de mirar, predice qué cabeceras nuevas llevará el segundo GET y con qué valores. Comprueba las cabeceras `If-Modified-Since` y relaciónalas con las cabeceras `Last-Modified` de la primera respuesta.

Cuestión 10.
¿Qué responde el servidor al segundo GET? ¿Lleva cuerpo? ¿Cuántos bytes se ahorran respecto a la primera respuesta? La primera respuesta traía `Cache-Control: no-cache`: ¿qué le pide eso al navegador y por qué hace que veamos el GET condicional?

Cookies (`cookies/`)

> **Antes de abrir la parte 7**
> La página se carga y se recarga. HTTP no guarda estado: ¿cómo podría saber el servidor, en la segunda petición, que es el mismo navegador?

Cuestión 11.
¿Qué cabecera trae la primera respuesta y qué cabecera lleva la segunda petición? ¿Por qué la segunda respuesta ya no trae `Set-Cookie`? Comprueba que la cookie viaja también en las peticiones de las partes 8 y 10: ¿por qué?

Cuestión 12.
¿Qué pasaría si visitaras la página en una ventana privada o después de borrar las cookies?

Autenticación (`protegido/`)

Cuestión 13.
¿Qué código y qué cabecera lleva la primera respuesta? ¿Qué cabecera añade el GET siguiente? Decodifica su valor (Base64): ¿está cifrado? Más adelante el navegador envía una contraseña incorrecta: ¿qué responde el servidor?

Cuestión 14.
Si pidieras otra página de la misma zona protegida, ¿volvería el navegador a enviar `Authorization`? ¿Qué tiene eso en común con la cookie de la parte 7?

Cuestión 15.
Después, el navegador pide la zona del profesorado (`protegido/profesor/`) con las mismas credenciales y recibe 403; con la contraseña incorrecta había recibido 401. ¿Qué distingue los dos códigos? ¿En cuál tiene sentido volver a pedir la contraseña al usuario?

4. De vuelta a tu navegador: inicio de sesión por formulario

Ya has visto dos formas de que el servidor reconozca al cliente aunque HTTP no guarde estado: una cookie que el servidor asigna sin preguntar nada (parte 7) y la autenticación básica, en la que el navegador pide la contraseña y la reenvía en cada petición (parte 8). La forma más habitual en la web es la tercera: un formulario de inicio de sesión que, si la contraseña es correcta, responde con una cookie de sesión. Vas a capturarla con tu navegador para compararla con las dos anteriores.

Con Wireshark capturando (o con una captura nueva) y “Desactivar caché” marcada, abre esta página y sigue este orden (usuario alumno, contraseña redes):

- `formulario.html`: **`http://lab.redesuah.win/lab/formulario.html`**
- Entra con GET (“Entrar con GET”) y pulsa “Zona privada”.
- Pulsa “Salir” y, después, otra vez “Zona privada”.
- Entra con POST (“Entrar con POST”) con una contraseña incorrecta y, después, con la correcta; pulsa “Zona privada”.

Filtra con `http.host == "lab.redesuah.win"` o con `http.cookie || http.set_cookie`. Si tu captura falla, usa la parte 10 de la captura de referencia (desde t ≈ 182 s), que sigue este mismo orden.

Formulario, cookie de sesión y zona privada (`formulario.html`)

Cuestión 16.

Carga `http://lab.redesuah.win/lab/formulario.html`: verás dos formularios con dos campos cada uno (usuario alumno, contraseña redes). Entra primero con GET (botón “Entrar con GET”) y, más adelante en la secuencia, con POST (botón “Entrar con POST”). ¿Dónde viajan el usuario y la contraseña en cada caso? En el POST, localiza las cabeceras `Content-Type` y `Content-Length` de la petición y comprueba la longitud del cuerpo. ¿Con cuál de los dos métodos queda la contraseña en la URL (y, por tanto, en el historial del navegador y en el registro del servidor)? ¿Y dónde viajaba la contraseña en la autenticación básica de la parte 8?

Cuestión 17.

Tras entrar correctamente, el servidor te envía una cookie. Haz después, en este orden:

- Pulsa “Zona privada”.
- Pulsa “Salir”.
- Pulsa otra vez “Zona privada”.
- Envía el formulario con POST con una contraseña incorrecta.

¿Qué cabecera y qué valor trae la respuesta a la entrada correcta, y qué lleva la petición de la zona privada? ¿En qué se diferencia esta cookie de la id_ar1 de la parte 7 (fíjate en `Expires`)? ¿Qué hace el servidor al pedir la zona privada después de Salir y cómo lo consigue? Con la contraseña incorrecta, ¿qué código de estado devuelve y en qué se diferencia de la autenticación básica de la parte 8?

También puedes practicar con capturas generadas al azar, con corrección y explicación de cada error, en la web de la asignatura (`https://ar1.threatinteluah.com`), en las plantillas de HTTP: petición y respuesta, modo de las conexiones, caché y visitas repetidas, y proxies.

Anexo. Wireshark: columnas, filtros y opciones útiles

Añadir la columna `tcp.stream`

- Selecciona un paquete TCP, despliega Transmission Control Protocol en el panel de detalle, haz clic derecho sobre “Stream index” y elige “Aplicar como columna” (Apply as Column).
- Para quitar o reordenar columnas: clic derecho sobre la cabecera de la lista de paquetes → “Preferencias de columna”.

Otras opciones

- **Seguir el flujo:** clic derecho sobre un mensaje → Seguir → Flujo HTTP (o TCP) muestra la conversación completa de esa conexión como texto.
- **Gráfico de flujo:** Estadísticas → Gráfico de flujo dibuja los mensajes entre los equipos (navegador y servidores) en orden temporal.
- **Reensamblado:** Editar → Preferencias → Protocols → TCP → “Allow subdissector to reassemble TCP streams”. Activado (por defecto), Wireshark muestra cada mensaje HTTP completo en el último de sus segmentos.

Para ver…	Filtro
Todos los mensajes HTTP	`http`
Solo peticiones / solo respuestas	`http.request / http.response`
Respuestas con un código	`http.response.code == 304`
Una conexión TCP completa	`tcp.stream == 5`
Establecimientos de conexión (SYN del cliente)	`tcp.flags.syn == 1 && tcp.flags.ack == 0`
Cierres de conexión	`tcp.flags.fin == 1`
Peticiones a un sitio	`http.host == "images.redesuah.win"`
Mensajes con cookies	`http.cookie || http.set_cookie`
Consultas y respuestas DNS	`dns`
Mensajes en un intervalo de tiempo	`frame.time_relative >= 111 && frame.time_relative < 130`